

LLM & AI Engineering — Quick Revision

Everything learned so far, kept short and arranged in sequence.

1. LLM Fundamentals

Concept	Short explanation
Generative AI	Generates new content such as text, code, images, audio, or video.
LLM	A language model trained on large text datasets to understand and generate language.
Transformer	The core neural-network architecture used by modern LLMs; attention helps it model relationships between tokens.
Inference	Using the trained model to generate an output from an input.
Capabilities	Generation, summarization, translation, coding, extraction, classification, reasoning, and more.
Limitations	Hallucinations, context limits, imperfect reasoning, stale knowledge without tools, bias, and cost.

2. LLM Processing Flow

Concept	Short explanation
Core sequence	Prompt → Tokenizer → Tokens → Token IDs → Embeddings → Vectors → Transformer → Encoding/processing → Decoding → Predict next token → Response.
Important idea	The model generates output token by token; each generated token becomes part of the context used to predict the next one.

3. LLM APIs

Concept	Short explanation
API fundamentals	Your application sends instructions/messages to a model through an API and receives a response.
Request lifecycle	Frontend → Backend → Build messages/prompt → LLM API → Response → Process/validate → Frontend.
Message roles	System = behavior/rules; User = user's request; Assistant = previous AI response.
Response processing	Parse, validate, format, stream, store, log, handle errors, and use the output in application logic.

4. AI Application Architecture

Concept	Short explanation
Basic	React/Next.js → Node.js/NestJS → LLM API.
Production	Frontend → Backend → authentication/business logic → cache/database/vector DB/tools → LLM → validation → response.
Key idea	The LLM is one component of the application, not the whole application.

5. Embeddings & Vector Databases

Concept	Short explanation
Embedding	Converts text/data into a numerical vector representing semantic meaning.
Vector DB	Stores embeddings and finds semantically similar vectors.
Semantic search	Finds relevant information by meaning rather than exact wording.
RAG	Question → embedding → vector search → relevant chunks → LLM → grounded answer.
Semantic cache	Stores question/response pairs with embeddings so similar future questions may reuse a previous answer.

6. Conversation & Context Management

Concept	Short explanation
Context	System instructions + relevant chat history + current user message + retrieved data + tool results.
Context window	Maximum tokenized information the model can process in one request.
Memory	Short-term = current context; long-term = externally stored information retrieved when needed.
Management techniques	Sliding window, summarization, retrieval-based memory, and RAG.

7. Model Configuration

Concept	Short explanation
Model	Select the model based on capability, speed, cost, and context needs.
Temperature	Controls generation variation/randomness; lower generally means more deterministic output.
Max output tokens	Limits response length.
Top-p	Controls the probability mass from which candidate tokens are sampled.
Response format	Text, Markdown, JSON, or structured schema.
Streaming	Sends generated output incrementally for better perceived responsiveness.

8. Tool Calling & Function Calling

Concept	Short explanation
Tool calling	LLM requests an external function/API to get data or perform an action.
Flow	User → LLM → tool decision → backend executes tool → tool result → LLM → final answer.
Tool design	One responsibility, clear name, well-defined parameters, structured result, validation, permissions.
Examples	Weather API, database query, calendar, CRM, ERP, email, calculator.

9. Structured Outputs & Validation

Concept	Short explanation
Structured output	LLM response in a predictable machine-readable format such as JSON.
Validation	Check syntax, schema, required fields, types, and business rules before using the result.
Flow	LLM → structured output → parse → schema validation → business validation → application/database.

10. Hallucinations & Limitations

Concept	Short explanation
Hallucination	A convincing but incorrect, fabricated, or unsupported response.
Why	LLMs predict likely token sequences; they do not automatically verify every fact.
Reduce risk	Better prompts + RAG + tool calling + validation + human review for high-risk decisions.

11. Reliability & Operations

Concept	Short explanation
Rate limits	Limits on requests/tokens over time; exceeding them commonly produces HTTP 429.
Handling limits	Exponential backoff, queues, caching, batching, and prompt/token reduction.

Concept	Short explanation
Cost	Main drivers include input/output tokens, model choice, embeddings, vector DB, storage, and infrastructure.
Caching	Exact cache such as Redis avoids repeat calls; semantic cache can reuse results for similar questions.
Monitoring	Track requests, latency, tokens, cost, errors, tool success, cache hit rate, retrieval quality, and user feedback.
Debugging	Inspect prompt → context → retrieval → tool calls → model response → parsing/business logic.

12. Prompt Engineering

Concept	Short explanation
Prompt basics	A good prompt usually defines task + context + constraints + expected output.
Zero-shot	Task with no examples.
One-shot	Task with one example.
Few-shot	Task with multiple examples to teach a pattern.
Persona	Defines how the AI communicates, such as friendly or concise.
Role	Defines what expertise/function the AI should act as, such as Senior React Architect.
Design patterns	Instruction, role, context, step-by-step, output format, delimiter, template, and self-check patterns.
Prompt debugging	Iterate: write → run → inspect → identify issue → improve → retest.
Prompt evaluation	Measure correctness, relevance, completeness, consistency, format, hallucination, cost, and latency.

13. AI Evaluation & Observability Tools

Concept	Short explanation
LangChain	Application framework/orchestration layer for models, prompts, tools, retrieval, structured output, and agents.
LangSmith	Observability/evaluation layer for tracing, debugging, datasets, prompt tests, and production analysis.
Promptfoo	Automated prompt/model comparison and regression testing.
DeepEval	Automated evaluation framework for LLM quality metrics.
OpenAI Evals	Framework for benchmark-style model/prompt evaluations.

The Big Picture — One Flow

Build: React → Node.js/NestJS → Prompt Builder → LLM API → Response Processing → UI

Add knowledge: Documents → Chunking → Embeddings → Vector DB → Retriever → Prompt → LLM

Add actions: User → LLM → Tool/Function Call → Backend/API/DB → Tool Result → LLM → Final Response

Add reliability: Cache → Rate-limit handling → Token/cost monitoring → Logs → Traces → Evaluation

Add prompt quality: Role + Task + Context + Constraints + Examples + Output Format → Evaluate → Iterate

One-Line Memory Map

LLM generates → Embeddings represent meaning → Vector DB retrieves → RAG supplies knowledge → Tools perform actions → Structured outputs make results machine-readable → Validation makes them safer → Caching reduces repeated work → Monitoring/Evals make the system reliable → LangChain orchestrates → LangSmith observes/evaluates.

Quick Interview Map

Question	Short answer
What is an LLM?	A model that understands and generates language.
What is an embedding?	A numerical vector representing semantic meaning.
What is a Vector DB?	A database for storing/searching embeddings by similarity.
What is RAG?	Retrieve relevant external information, then give it to the LLM to generate a grounded answer.
What is Tool Calling?	The LLM requests a predefined function/API; the backend executes it.
What is a hallucination?	A plausible-looking but incorrect or unsupported answer.
What is LangChain?	A framework for orchestrating LLM application components.
What is LangSmith?	A platform for tracing, debugging, monitoring, and evaluating LLM applications.